HAFTA

04

CSS ve Sayfa Düzeni

Belgemiz var ama üst üste yığılmış duruyor. Bu hafta tarayıcıya neyin nerede ve nasıl duracağını anlatmayı öğreniyoruz.

BS435 · Web Programlama I

HAFTANIN SORUSU

Aynı HTML telefonda ve masaüstünde nasıl farklı görünüyor?

Bu haftanın hedefleri

Her başlığın derinliği aynı değil

MUTLAKA BİL

Basamaklanmayı, kutu modelini ve Flexbox ile uyarlanabilir düzen kurmayı açıkla.

DERSTE UYGULA

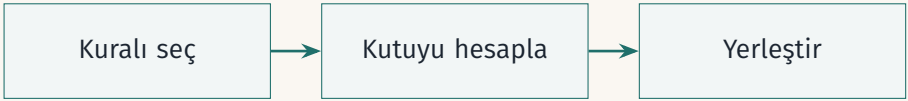
Çakışan bir CSS kuralını `F12` panelinde bul; düzeni üç genişlikte incele.

MERAK EDİYORSAN İNCELE

Özgüllük puanları, Grid, önce mobil yaklaşımı ve ayrıntılı erişilebilirlik ölçütleri.

Kampüs panosu: CSS

Aynı pano kartlarının telefonda alt alta, geniş ekranda yan yana görünmesini istiyoruz. HTML içeriği aynı kalıyor.



DÜŞÜN · 45 SANİYE

300 px içerik + iki yanda 20 px padding + 2 px kenarlık kaç px eder?

1

CSS nasıl karar verir?

Aynı öğeye üç farklı kural uyuyorsa hangisi kazanır?

Bir kuralın anatomisi

```
h1.baslik {              /* seçici (selector) */
    color: #C2532F;      /* özellik: değer → bildirim */
}
```

SEÇİCİ NEYİ SEÇER

- `p` tüm paragraflar
- `.kart` `class="kart"` olanlar
- `#menu` `id="menu"` olan

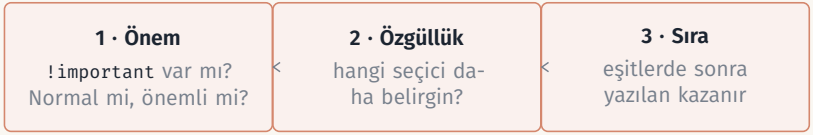
İÇ İÇE VE DURUMA GÖRE

- `nav a` nav içindeki tüm bağlantılar
- `nav > a` yalnızca doğrudan çocuklar
- `a:hover` üzerine gelindiğinde

CSS sayfaya üç yolla bağlanır: `<link>` ile ayrı dosya, `<style>` ile belge içi, `style="..."` ile öğe içi. Yeniden kullanılabilir stiller için `.class` seçin.

Cascade: üç soru, sırayla

Basitleştirilmiş model: aynı kaynak ve katmandaki kurallar



Bir de **kalıtım** var: `color` ve `font-family` gibi özellikler çocuklara geçer, `border` ve `margin` geçmez. Alt öğe kendi değerini belirlerse kalıtılan değeri kullanmaz.

Özgüllük nasıl sayılır?

SEÇİCİ	PUAN	AÇIKLAMA
`p`	0 – 0 – 1	element
`.kart`	0 – 1 – 0	sınıf
`#menu`	1 – 0 – 0	kimlik
`nav .kart p`	0 – 1 – 2	birleşik
`style="..."`	normal yazar kurallarını aşar	satır içi

NOT

Soldan sağa karşılaştırılır: aynı öncelik düzeyinde `1-0-0`, `0-9-9`'u yener.

Neden benim kuralım uygulanmıyor?

>_ CANLI DEMO

Aynı öğeye kasten çakışan üç kural yazalım ve **F12 → Styles** panelinde hangisinin üstü çizildiğini birlikte okuyalım.

- Üstü çizili bildirim **kaybetmiş** demektir — neden kaybettiğini soralım.
- Çözüm neredeyse hiçbir zaman `!important` değildir.
- Doğru refleks: **seçiciyi sadeleştirmek**, özgüllük yarışına girmemek.

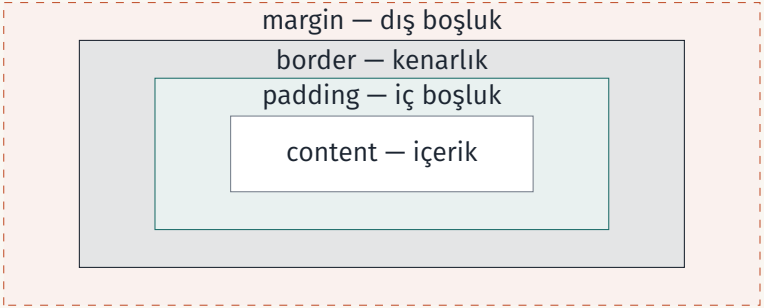
→ **WP-II** Bu panel, WP-II'de React bileşenlerinin stilini ayıklarken en çok kullanacağınız araç olacak.

2

Kutu modeli

CSS’te her şey bir kutudur — metin de, görsel de, sayfanın tamamı da.

Kutu modeli: dört katman



Arka plan padding alanına uzanır; margin şeffaftır. Normal blok akışında alt alta gelen **dikey margin değerleri birleşebilir** — flex ve grid öğelerinde birleşmez.

Genişlik neyi kapsıyor?

VARSAYILAN

```
width: 300px;
padding: 20px;
border: 2px solid;

/* gercek genislik:
   300 + 40 + 4 = 344px */
```

BORDER-BOX

```
box-sizing: border-box;
width: 300px;
padding: 20px;
border: 2px solid;

/* gerçek genişlik: 300px */
```

YAYGIN PRATİK

Çoğu proje en başta `*{ box-sizing: border-box; }` yazar.

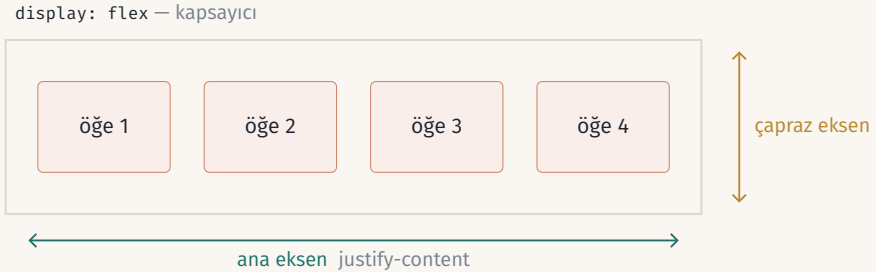
3

Düzen

Kutuları yan yana dizmenin üç kuşağı: akış, flexbox, grid.

Flexbox — tek eksende dizme

Normal akışta **block** öğeler (`div`, `p`, `h1`) satırın tamamını kaplayıp alt alta, **inline** öğeler (`a`, `span`) yalnızca içerikleri kadar yer kaplayıp yan yana dizilir. Dizilimi biz yönetmek istediğimizde kapsayıcıya `display: flex` veririz.



- `justify-content` ana eksende dağıtır (`space-between`, `center`)
- `align-items` çapraz eksende hizalar (`center`, `stretch`)
- `gap` öğeler arasındaki boşluğu doğrudan tanımlar

Dar ekrandan geniş ekrana

Önce mobil (mobile-first)

```
.kart-listesi {
    display: flex;
    flex-wrap: wrap;        /* sığmayınca alt satıra geç */
    gap: 16px;
}
.kart { flex: 1 1 240px; }  /* büyü, küçül, 240px'ten başla */
```

- Kartlar geniş ekranda yan yana, dar ekranda alt alta dizilir — **tek bir medya sorgusu yazmadan**.
- Medya sorgusu, dizilim değil **düzenin kendisi** değişecekse gerekir: dar ekranı varsayılan yazıp `@media (min-width: 768px)` ile üstüne eklersiniz.
- `<meta name="viewport" ...>` satırı olmadan telefon sayfayı cihaz genişliğine uyarlamaz.

→ **WP-II** Grid, iki eksende birden düzen kurar. Sayfa iskeleti için grid, bileşen içi dizilim için flexbox iyi bir varsayılandır.

4

Erişilebilirlik

Sayfanız yalnızca sizin gördüğünüz koşullarda kullanılmayacak.

Erişilebilir arayüzler

- **Kontrast** metin ve arka plan arasında yeterli fark olmalı — tasarım tercihi değil, okunabilirlik şartı.
- **Odak görünürlüğü** klavyeyle gezenler için `:focus` halkasını silmeyin; değiştirin ama kaldırmayın.

Aynı başlığın altında iki şey daha var: tıklanabilir alanı parmakla rahat vurulacak kadar büyük tutmak ve rengi tek başına anlam taşıyıcı yapmamak — “kırmızı olanlar hatalı” demek, renk körü kullanıcıya hiçbir şey söylemez.

Sıra sizde

ALIŞTIRMA · 5 dk · tarayıcıda

Açık duran kart listesinde **F12 → cihaz araç çubuğu** ile genişliği değiştirin ve her adımda bakın:

1. **360 px** — kartlar tek sütuna düşüyor mu?
2. **768 px** — kaç sütun oldu, aradaki boşluk korundu mu?
3. **1280 px** — yatay kaydırma çubuğu çıkıyor mu?

Listeyi bu davranışa kavuşturan CSS'i yazma işi haftalık uygulamada: `examples/hafta-04-css`.

Haftanın sorusunun cevabı

HTML hiç değişmedi. Değişen tek şey, tarayıcının **kutuları nasıl dizeceğine** dair talimattı.

- **Cascade** hangi kuralın kazanacağına karar verdi.
- **Kutu modeli** her öğenin ne kadar yer kapladığını belirledi.
- **Flexbox ve medya sorguları** aynı kutuları farklı genişliklerde farklı dizdi.

> **ÇIKIŞ BİLETİ · HER YANITA BİR GEREKÇE**
>
> Özgüllük eşitse son kural her koşulda kazanır mı? `border-box` margin'i genişliğe dahil eder mi? Flex öğelerinin alt satıra geçmesini hangi kural açar?

Nerede kaldık?

Dört hafta sonra: sayfa artık görünüyor

01 Ağ ve DNS	**02** HTTP	**03** HTML ve DOM	**04** CSS
05 Formlar	**06** Git ve yayın	**07** JavaScript	**08** DOM ve olaylar
09 Asenkron ve JSON	**10** Fetch ve API	**11** Durum ve önbellek	**12** Backend
13 Veritabanı	**14** Güvenlik		

Kiremit çerçeve: işlendi. Gri çerçeve: sonraki haftalar.

Önümüzdeki hafta

Sayfayı gördük. Peki kullanıcı bir şey **yazıp gönderdiğinde** sunucuya hangi veriler gidiyor?

Formları işleyeceğiz — ve 2. haftada gördüğümüz HTTP isteğinin bir formdan nasıl doğduğunu izleyeceğiz.

> **→ WEB PROGRAMLAMA II**
>
> Cascade ve kutu modeli, WP-II'de React bileşenlerine stil verirken aynen geçerli olacak.